

Pasión por la Tecnología



Fundamentos de Gestión de Datos

Docente:
Pilar Rocío Sayán Mejía

LABORATORIO N° 06

Semana 6:

*Limpieza de Datos sobre
Tabla Desnormalizada*



LABORATORIO DIRIGIDO N° 06 Semana 6: Limpieza de Datos sobre Tabla Desnormalizada- abarrotes_ventas_inventario.db	
DOCENTE: Pilar Rocío Sayán Mejía	CURSO: Fundamentos de Gestión de Datos

I. Objetivo del Laboratorio

Implementar un pipeline de limpieza de datos sobre la tabla desnormalizada `ventas_original`, aplicando: diagnóstico de calidad, corrección de formatos, imputación de valores faltantes, tratamiento de duplicados, detección y tratamiento de outliers con IQR, normalización Min-Max, y exportación del dataset limpio como nueva tabla en SQLite.

II. Elemento de la Capacidad Terminal

Construye un pipeline básico de limpieza de datos con Python y Pandas, cargando información desde SQLite, diagnosticando problemas de calidad (nulos, tipos, duplicados y outliers), aplicando funciones de limpieza, imputación y normalización Min-Max, para exportar un dataset limpio y reutilizable.

III. Seguridad

- Mantener una postura adecuada durante el trabajo en computadora.
- Guardar avances de forma periódica en Google Colab o en el entorno local.
- No modificar la base original; crear copias de trabajo para las transformaciones.
- Verificar que el enlace del repositorio de GitHub sea público antes de ejecutar la descarga.

IV. Fundamento Teórico

La limpieza de datos permite preparar información para análisis confiable. En bases desnormalizadas es frecuente encontrar inconsistencias de formato (texto sucio, fechas mal tipadas, números como texto), valores faltantes, registros duplicados y valores atípicos (outliers). Un pipeline de limpieza debe diagnosticar, corregir y validar cada paso, dejando constancia del antes y después de cada transformación relevante.

V. Normas empleadas

- Usar nombres de variables claros y consistentes en Python.
- Separar el diagnóstico, la limpieza y la validación final en celdas distintas.
- Conservar evidencia del antes y después de cada transformación relevante.
- No ocultar problemas de calidad: deben explicarse y justificarse en las respuestas.

VI. Recursos

- Google Colab con cuenta Google activa.
- Base SQLite: abarrotes_ventas_inventario.db, descargada desde GitHub.
- Tabla principal: ventas_original.
- Librerías: pandas, numpy, sqlite3, requests, matplotlib y seaborn.

Metodología

- Trabajo dirigido en clase con ejecución paso a paso.
- El estudiante debe ejecutar todas las celdas, interpretar resultados y responder las preguntas.
- La base original no se reemplaza; se crea una tabla limpia llamada ventas_original_limpia_s6.
- Al finalizar, entregar el notebook con salidas visibles y el archivo generado.

VII. Procedimiento

Caso: Abarrotes Esperanza cuenta con una tabla desnormalizada de ventas. En la Semana 6 se debe preparar esta información para análisis. El pipeline parte de ventas_original (no de tablas normalizadas).

Revisión de conocimientos de la semana

Antes de ejecutar el código, completa el siguiente cuadro con tus propias palabras.

Concepto / Principio	Definición — responde con tus propias palabras
1. Pipeline de limpieza de datos	
2. Valores faltantes e imputación	
3. Outliers y método IQR	
4. Normalización Min-Max	
5. Tabla desnormalizada	
6. Variabilidad	
7. Mediana	
8. Desviación estándar	
9. Cuartiles Q2 y Q3	
10. Rango intercuartil (IQR)	
11. Técnicas de imputación	

Técnicas de imputación de valores faltantes

Cuando una columna tiene valores ausentes, existen varias estrategias para completarlos sin perder registros. La imputación por la media es útil en variables numéricas con distribución simétrica; la mediana es preferible cuando hay valores atípicos (outliers) que distorsionarían el promedio; la moda se aplica en variables categóricas o discretas. También se puede usar un valor constante (por ejemplo, 0 o "Sin dato"), imputación hacia adelante o hacia atrás (para series de tiempo), o modelos predictivos como KNN. En este laboratorio aplicaremos mediana para numéricas y moda para categóricas.

Actividad 1. Carga de librerías y base de datos desde GitHub

1. Importar las librerías: requests, sqlite3, pandas, numpy, warnings, matplotlib, seaborn.
2. Descargar la base SQLite desde el repositorio GitHub usando requests.
3. Conectarse con sqlite3 y verificar la conexión.
4. Mostrar las primeras 5 filas de ventas_original.

```

try:
    import requests
except ModuleNotFoundError:
    requests = None

import sqlite3
import pandas as pd
import numpy as np
import warnings
try:
    import matplotlib.pyplot as plt
    import seaborn as sns
except ModuleNotFoundError:
    plt = None; sns = None

url =
"https://raw.githubusercontent.com/Rociosayan/PMD1_Fundamentos_Gestion_Datos/main/casos/16_abarros_vendas_inventario/abarros_vendas_inventario.db"

if requests is not None:
    r = requests.get(url)
    r.raise_for_status()
    contenido = r.content
else:
    from urllib.request import urlopen
    with urlopen(url) as respuesta:
        contenido = respuesta.read()

open("abarros_vendas_inventario.db", "wb").write(contenido)
conn = sqlite3.connect("abarros_vendas_inventario.db")
df = pd.read_sql_query("SELECT * FROM vendas_original LIMIT 5", conn)
df

```

► Ejecuta el código sugerido en el notebook y responde la pregunta.

Pregunta 1. ¿Por qué en la Semana 6 se trabaja desde `vendas_original` y no desde tablas normalizadas?

Respuesta:

Actividad 2. Carga completa de `vendas_original`

1. Leer la tabla `vendas_original` completa con `pd.read_sql_query()`.
2. Mostrar las primeras filas con `.head()`.

```

vendas_original = pd.read_sql_query("SELECT * FROM vendas_original;", conn)
vendas_original.head()

```

► Ejecuta el código sugerido en el notebook y responde la pregunta.

Pregunta 2. ¿Qué columnas presentan mayor riesgo de calidad y por qué?

Respuesta:

Actividad 3. Diagnóstico inicial de calidad

1. Mostrar dimensiones con `.shape` e información con `.info()`.
2. Construir tabla resumen: tipo de dato, nulos, porcentaje de nulos y valores únicos.
3. Ordenar por porcentaje de nulos descendente.

```
print("Filas y columnas:", ventas_original.shape)
display(ventas_original.info())

diagnostico = pd.DataFrame({
    "tipo_dato": ventas_original.dtypes.astype(str),
    "nulos": ventas_original.isna().sum(),
    "porcentaje_nulos": (ventas_original.isna().mean() * 100).round(2),
    "unicos": ventas_original.nunique(dropna=True)
})
diagnostico.sort_values("porcentaje_nulos", ascending=False)
```

► Ejecuta el código sugerido en el notebook y responde la pregunta.

Pregunta 3. ¿Qué señales muestran que la tabla está desnormalizada?

Respuesta:

Actividad 4. Revisión de duplicados y consistencia básica

1. Calcular duplicados exactos con `.duplicated().sum()`.
2. Revisar valores únicos de columnas de texto con `.nunique()`.

```
duplicados = ventas_original.duplicated().sum()
print("Duplicados exactos:", duplicados)

columnas_texto = ventas_original.select_dtypes(include="object").columns
resumen_texto =
ventas_original[columnas_texto].nunique().sort_values(ascending=False)
resumen_texto
```

► Ejecuta el código sugerido en el notebook y responde la pregunta.

Pregunta 4. ¿Qué problemas pueden aparecer si no se corrigen formatos de texto, fecha y número?

Respuesta:

Actividad 4a. Estadísticas descriptivas e histogramas

1. Calcular estadísticas descriptivas (media, desviación estándar, percentiles) con `describe()`.
2. Graficar un histograma por cada variable numérica relevante.
3. Identificar si la distribución es simétrica, sesgada o bimodal.

```
# Estadísticas descriptivas de variables numéricas
ventas_original.describe().round(2)
# Variables numéricas para visualizar
cols_num = [c for c in [
    "precio_venta_unitario", "costo_unitario", "cantidad_unidades",
    "monto_venta_soles", "margen_venta_soles"
] if c in ventas_original.columns]

# Histogramas
fig, axes = plt.subplots(len(cols_num), 1, figsize=(10, 4*len(cols_num)))
if len(cols_num) == 1: axes = [axes]
for ax, col in zip(axes, cols_num):
    ventas_original[col].dropna().hist(ax=ax, bins=30, color="steelblue",
    edgecolor="white")
    ax.set_title(f"Histograma — {col}", fontsize=11)
    ax.set_xlabel(col); ax.set_ylabel("Frecuencia")
plt.tight_layout()
plt.show()
```

► Ejecuta el código sugerido en el notebook y responde la pregunta.

Pregunta 4a. ¿Qué distribución predomina en las variables numéricas? ¿Hay alguna con alto sesgo?

Respuesta:

Actividad 4b. Diagramas de cajas y detección visual de outliers

1. Generar un boxplot por cada variable numérica.
2. Identificar los puntos fuera de los bigotes: son los posibles outliers.
3. Registrar qué variables presentan mayor cantidad de valores atípicos.

```
# Diagrama de cajas — los puntos fuera de los bigotes son OUTLIERS
fig, axes = plt.subplots(1, len(cols_num), figsize=(4*len(cols_num), 5))
```

```

if len(cols_num) == 1: axes = [axes]
for ax, col in zip(axes, cols_num):
    ventas_original.boxplot(column=col, ax=ax)
    ax.set_title(col, fontsize=9)
plt.suptitle("Boxplots — variables continuas (puntos = outliers)", y=1.02)
plt.tight_layout()
plt.show()

```

► Ejecuta el código sugerido en el notebook y responde la pregunta.

Pregunta 4b. ¿Cuáles variables muestran más outliers en el boxplot? ¿Cómo se relaciona esto con el método IQR de la Actividad 9?

Respuesta:

Actividad 4c. Gráficas de barras y diagrama de pie

1. Graficar la frecuencia de cada categoría en las variables de texto.
2. Identificar categorías dominantes o con muy poca representación.
3. Generar un diagrama de pie para la variable categórica principal.

```

# Variables categóricas (máx. 15 valores únicos para graficar bien)
cols_cat = [c for c in ventas_original.select_dtypes(include="object").columns
             if ventas_original[c].nunique() <= 15]

# Gráficas de barras
fig, axes = plt.subplots(len(cols_cat), 1, figsize=(10, 4*len(cols_cat)))
if len(cols_cat) == 1: axes = [axes]
for ax, col in zip(axes, cols_cat):
    counts = ventas_original[col].value_counts().head(10)
    counts.plot.bar(ax=ax, color="steelblue", edgecolor="white")
    ax.set_title(f"Distribución — {col}", fontsize=11)
    ax.set_ylabel("Frecuencia"); ax.tick_params(axis="x", rotation=45)
plt.tight_layout()
plt.show()

# Diagrama de pie — primera variable categórica
if cols_cat:
    col_pie = cols_cat[0]
    counts = ventas_original[col_pie].value_counts().head(8)
    fig, ax = plt.subplots(figsize=(7, 5))
    counts.plot.pie(ax=ax, autopct="%1.1f%%", startangle=90)
    ax.set_title(f"Proporción — {col_pie}", fontsize=12)
    ax.set_ylabel("")
    plt.tight_layout()
    plt.show()

```

► Ejecuta el código sugerido en el notebook y responde la pregunta.

Pregunta 4c. ¿Qué categoría predomina y qué porcentaje del total representa?

Respuesta:

Actividad 4d. Tablas cruzadas

1. Construir una tabla cruzada (crosstab) entre dos variables categóricas.
2. Visualizar la tabla como mapa de calor para detectar patrones.
3. Interpretar qué combinación de categorías aparece con mayor frecuencia.

```
# Tabla cruzada entre dos variables categóricas
if len(cols_cat) >= 2:
    tabla_cruzada = pd.crosstab(
        ventas_original[cols_cat[0]],
        ventas_original[cols_cat[1]],
        margins=True
    )
    print("=== TABLA CRUZADA ===")
    display(tabla_cruzada)

# Mapa de calor de la tabla cruzada
fig, ax = plt.subplots(figsize=(10, 6))
import seaborn as sns
sns.heatmap(tabla_cruzada.iloc[:-1, :-1], annot=True, fmt="d",
            cmap="Blues", ax=ax)
ax.set_title("Mapa de calor — Tabla cruzada")
plt.tight_layout()
plt.show()
```

► Ejecuta el código sugerido en el notebook y responde la pregunta.

Pregunta 4d. ¿Qué relación encontraron entre las dos variables categóricas del crosstab?

Respuesta:

Actividad 5. Funciones de limpieza

1. Definir limpiar_texto(): quitar espacios, normalizar mayúsculas con .title().
2. Definir convertir_numero(): usar pd.to_numeric con errors='coerce'.
3. Definir convertir_fecha_mixta(): manejar formatos mixtos con pd.to_datetime.

```
def limpiar_texto(serie):
```

```

return (
    serie.astype("string")
    .str.strip()
    .str.replace(r"\s+", " ", regex=True)
    .str.title()
)

def convertir_numero(serie):
    return pd.to_numeric(serie, errors="coerce")

def convertir_fecha_mixta(serie):
    with warnings.catch_warnings():
        warnings.simplefilter("ignore", UserWarning)
        fecha_1 = pd.to_datetime(serie, errors="coerce", dayfirst=False)
        fecha_2 = pd.to_datetime(serie, errors="coerce", dayfirst=True)
    return fecha_1.fillna(fecha_2)

```

► Ejecuta el código sugerido en el notebook y responde la pregunta.

Pregunta 5. Justifique una decisión de imputación aplicada en el laboratorio.

Respuesta:

Actividad 6. Corrección de tipos y formatos

1. Crear copia del DataFrame: `df_limpio = ventas_original.copy()`.
2. Aplicar `limpiar_texto()` a texto y `convertir_fecha_mixta()` a fechas.
3. Aplicar `convertir_numero()` a columnas numéricas.
4. Verificar tipos resultantes con `.dtypes`.

```

df_limpio = ventas_original.copy()

for col in df_limpio.select_dtypes(include="object").columns:
    if "fecha" in col.lower():
        df_limpio[col] = convertir_fecha_mixta(df_limpio[col])
    else:
        df_limpio[col] = limpiar_texto(df_limpio[col])

columnas_numericas = [
    "cantidad_unidades", "precio_venta_unitario", "costo_unitario",
    "descuento_pct", "stock_inicial", "stock_final", "merma_unidades",
    "monto_venta_soles", "margen_venta_soles",
    "productos_costo_unitario", "productos_precio_lista"
]

for col in columnas_numericas:
    if col in df_limpio.columns:
        df_limpio[col] = convertir_numero(df_limpio[col])

```

df_limpio.dtypes

► Ejecuta el código sugerido en el notebook y responde la pregunta.

Pregunta 6. ¿Qué diferencia hay entre eliminar duplicados y corregir inconsistencias de contenido?

Respuesta:

Actividad 7. Imputación de valores faltantes

1. Imputar variables numéricas con la mediana de cada columna.
2. Imputar variables categóricas con la moda o con la etiqueta "Sin Dato".
3. Imputar fechas con la mediana temporal.
4. Comparar nulos antes y después de la imputación.

```
nulos_antes = df_limpio.isna().sum()

for col in df_limpio.select_dtypes(include=["number"]).columns:
    mediana = df_limpio[col].median()
    df_limpio[col] = df_limpio[col].fillna(mediana)

for col in df_limpio.select_dtypes(include=["string", "object"]).columns:
    moda = df_limpio[col].mode(dropna=True)
    valor = moda.iloc[0] if len(moda) > 0 else "Sin Dato"
    df_limpio[col] = df_limpio[col].fillna(valor)

for col in df_limpio.select_dtypes(include=["datetime64[ns]").columns:
    mediana_fecha = df_limpio[col].dropna().median()
    if pd.isna(mediana_fecha):
        mediana_fecha = pd.Timestamp("1900-01-01")
    df_limpio[col] = df_limpio[col].fillna(mediana_fecha)

nulos_despues = df_limpio.isna().sum()
pd.DataFrame({
    "nulos_antes": nulos_antes,
    "nulos_despues": nulos_despues,
    "nulos_corregidos": nulos_antes - nulos_despues
}).query("nulos_antes > 0 or nulos_despues > 0")
```

► Ejecuta el código sugerido en el notebook y responde la pregunta.

Pregunta 7. ¿Por qué se usa IQR para detectar outliers y cuándo no conviene eliminar registros?

Respuesta:

Actividad 8. Tratamiento de duplicados

1. Registrar número de filas antes de eliminar duplicados.
2. Aplicar `.drop_duplicates()` y reiniciar el índice.
3. Mostrar filas antes, después y duplicados eliminados.

```
filas_antes = len(df_limpio)
df_limpio = df_limpio.drop_duplicates().reset_index(drop=True)
filas_despues = len(df_limpio)

print("Filas antes:", filas_antes)
print("Filas después:", filas_despues)
print("Duplicados eliminados:", filas_antes - filas_despues)
```

► Ejecuta el código sugerido en el notebook y responde la pregunta.

Pregunta 8. ¿Para qué sirve normalizar variables numéricas en una etapa posterior de análisis?

Respuesta:

Actividad 9. Detección y tratamiento de outliers con IQR

1. Seleccionar las columnas numéricas relevantes.
2. Calcular Q1, Q3 e IQR para cada columna.
3. Detectar valores fuera de $[Q1 - 1.5 \times IQR, Q3 + 1.5 \times IQR]$.
4. Capar valores extremos con `.clip()` y generar reporte de outliers detectados.

```
columnas_outliers = [
    col for col in [
        "cantidad_unidades", "precio_venta_unitario", "costo_unitario",
        "descuento_pct", "stock_inicial", "stock_final", "merma_unidades",
        "monto_venta_soles", "margen_venta_soles",
        "productos_costo_unitario", "productos_precio_lista"
    ]
    if col in df_limpio.columns
]

reporte_outliers = []
for col in columnas_outliers:
    q1 = df_limpio[col].quantile(0.25)
```

```

q3 = df_limpio[col].quantile(0.75)
iqr = q3 - q1
lim_inf = q1 - 1.5 * iqr
lim_sup = q3 + 1.5 * iqr
mascara = (df_limpio[col] < lim_inf) | (df_limpio[col] > lim_sup)
reporte_outliers.append([col, int(mascara.sum()), lim_inf, lim_sup])
df_limpio[col] = df_limpio[col].astype(float).clip(lower=lim_inf, upper=lim_sup)

pd.DataFrame(reporte_outliers,
              columns=["columna", "outliers_detectados", "limite_inferior", "limite_superior"])

```

► Ejecuta el código sugerido en el notebook y responde la pregunta.

Pregunta 9. Interprete el reporte antes/después: ¿la calidad del dataset mejoró? Sustente.

Respuesta:

Actividad 10. Normalización Min-Max

1. Crear copia df_normalizado a partir de df_limpio.
2. Aplicar la fórmula Min-Max a cada columna numérica limpia.
3. Verificar que las columnas _minmax estén en [0, 1] con .describe().

```

df_normalizado = df_limpio.copy()

for col in columnas_outliers:
    minimo = df_normalizado[col].min()
    maximo = df_normalizado[col].max()
    if maximo != minimo:
        df_normalizado[col + "_minmax"] = (df_normalizado[col] - minimo) / (maximo - minimo)
    else:
        df_normalizado[col + "_minmax"] = 0

df_normalizado[
    [col for col in df_normalizado.columns if col.endswith("_minmax")]
].describe().round(3)

```

► Ejecuta el código sugerido en el notebook y responde la pregunta.

Pregunta 10. ¿Qué tabla queda creada en SQLite y cómo se podría usar en el siguiente laboratorio?

Respuesta:

Actividad 11. Reporte antes/después

1. Construir tabla comparativa: filas, columnas, nulos totales, duplicados exactos.
2. Comparar ventas_original vs. df_normalizado.

```
reporte_final = pd.DataFrame({
    "metrica": ["filas", "columnas", "nulos_totales", "duplicados_exactos"],
    "antes": [
        ventas_original.shape[0],
        ventas_original.shape[1],
        int(ventas_original.isna().sum().sum()),
        int(ventas_original.duplicated().sum())
    ],
    "despues": [
        df_normalizado.shape[0],
        df_normalizado.shape[1],
        int(df_normalizado.isna().sum().sum()),
        int(df_normalizado.duplicated().sum())
    ]
})
reporte_final
```

► Ejecuta el código sugerido en el notebook y responde la pregunta.

Pregunta 11. ¿Cómo demostrarías que la limpieza mejoró la calidad del dataset? ¿Qué métricas son las más relevantes?

Respuesta:

Actividad 12. Exportación del dataset limpio

1. Guardar df_normalizado como tabla ventas_original_limpia_s6 en SQLite (if_exists='replace').
2. Exportar también a CSV con encoding utf-8-sig.
3. Validar con un SELECT COUNT(*) desde SQLite.

```
df_normalizado.to_sql(
    "ventas_original_limpia_s6", conn, if_exists="replace", index=False
)
df_normalizado.to_csv(
    "ventas_original_limpia_s6.csv", index=False, encoding="utf-8-sig"
)

validacion = pd.read_sql_query(
```

```
"SELECT COUNT(*) AS filas_limpias FROM ventas_original_limpia_s6;", conn  
)  
validacion
```

► *Ejecuta el código sugerido en el notebook y responde la pregunta.*

Pregunta 12. ¿Por qué es importante guardar el dataset limpio como tabla nueva en lugar de reemplazar la original?

Respuesta:

VIII. Evidencias a entregar

- Notebook del Laboratorio Dirigido N.º 06 ejecutado de inicio a fin.
- Capturas o salidas visibles del diagnóstico inicial y del reporte antes/después.
- Archivo ventas_original_limpia_s6.csv generado por el notebook.
- Respuestas desarrolladas en las 12 preguntas del laboratorio.

IX. Conclusiones

Conclusión 1:

Conclusión 2:

Conclusión 3:

X. Rúbrica holística de evaluación

		Fundamentos de Gestión de Datos				
		Rúbrica holística				
Elemento de la capacidad	Implementar un pipeline de limpieza de datos sobre la tabla desnormalizada ventas_original, aplicando					
Curso	Fundamentos de Gestión de Datos				Periodo	2026-I
Actividad	Laboratorio Dirigido 6				Semestr e	1er
Nombre del alumno					Semana	6
Docente		Fecha			Sección	AB
Criterios a evaluar		Excelente	Bueno	Requiere mejora	No aceptabl e	Puntaje logrado
Criterio 1: Carga y diagnóstico (Act. 1–3)		Carga desde GitHub, muestra shape/info, tabla de nulos ordenada y detecta duplicados.	Carga la base y presenta diagnóstico básico con alguna observación faltante.	Carga la base, pero el diagnóstico es incompleto o sin ordenar.	No carga la base o no presenta diagnóstico.	4
Criterio 2: Limpieza e imputación (Act. 4–7)		Aplica las tres funciones, imputa nulos con mediana/moda, verifica antes/después correctamente.	Aplica la mayoría de funciones con alguna imperfección menor en la imputación.	Aplica parcialmente las funciones o la imputación presenta errores.	No aplica funciones de limpieza ni imputa nulos.	4
Criterio 3: Duplicados y outliers (Act. 8–9)		Elimina duplicados, aplica IQR con clip y genera reporte	Elimina duplicados y detecta outliers, pero el reporte es parcial.	Solo trata duplicados o solo detecta outliers, sin completar ambos pasos.	No trata duplicados ni outliers.	4

	completo de outliers.				
Criterio 4: Normalización y exportación (Act. 10–12)	Aplica Min-Max, verifica rango [0,1] y exporta a SQLite y CSV con SELECT de validación.	Normaliza y exporta correctamente, con alguna observación menor.	Normalización o exportación incompleta o sin validación.	No normaliza ni exporta el dataset limpio.	4
Criterio 5: Preguntas reflexivas (Preg. 1–12)	Las 12 preguntas respondidas con argumentos técnicos precisos y justificación clara.	Al menos 9 preguntas con argumentos aceptables.	Menos de 9 preguntas respondidas o respuestas superficiales.	No responde las preguntas o deja espacios vacíos.	4
Total					

<i>Acciones a cumplir</i>	<i>Menos</i>
1. Puntualidad y dedicación	1
2. Cumplimiento de tiempos establecidos	1
3. Conclusiones: ortografía y redacción	1
<i>Puntaje total</i>	

Comentarios respecto del desempeño del alumno	
---	--

	<i>Descripción</i>
Excelente	Demuestra un completo entendimiento del problema o realiza la actividad cumpliendo todos los requerimientos especificados.
Bueno	Demuestra un considerable entendimiento del problema o realiza la actividad cumpliendo con la mayoría de los requerimientos especificados.
Requiere mejora	Demuestra un bajo entendimiento del problema o realiza la actividad con pocos de los requerimientos especificados.
No aceptable	No demuestra entendimiento del problema o actividad.